HOUSEHOLD ANIMALS

CLASSIFICATION USING DEEP LEARNING

A comparative study of CNN training, regularization, transfer learning and fine-tuning

PROJECT AUTHOR	TEAMMATE
Anish A Kunder SRN: PES1UG24AM343	**Adarsha E** SRN: PES1UG24AM334

ABSTRACT

This project develops a binary image classifier that predicts whether an uploaded image contains a cat (label 0) or a dog (label 1). The study compares a VGG-style convolutional neural network trained from scratch with regularization, ImageNet-pretrained VGG-16 feature extraction, and VGG-16 fine-tuning. The workflow includes reproducible data splits, held-out testing, model checkpoint selection using validation loss, Grad-CAM explanations, and an interactive Streamlit demo. On the shared test set, the best observed accuracy was 99.50% for the fine-tuned comparison model.

1. PROBLEM STATEMENT AND OBJECTIVE

Manual image sorting into cats and dogs is a straightforward image-recognition task that provides a useful setting to study model generalization. The objective is to build and compare deep-learning approaches under a consistent evaluation protocol, quantify the effect of regularization and transfer learning, and provide a visual explanation of predictions. The model is restricted to two labels; it does not recognize other animal categories.

2. DATASET AND PREPARATION

The project uses the Kaggle Dogs vs. Cats labeled dataset, initially containing 25,000 images (12,500 cats and 12,500 dogs). Each image is checked for readability and its filename-derived class. Exact duplicates are identified by hashing decoded RGB pixels; six images are excluded, with reasons recorded in excluded.csv. A stratified split with seed 42 yields 24,994 images: 19,994 for training, 3,998 for validation, and 1,002 for testing. The split script verifies that image paths and exact RGB hashes do not cross splits.

Split	Cats	Dogs	Total
Train: 80%	9,997	9,997	19,994
Validation: 16%	1,999	1,999	3,998
Test: 4%	501	501	1,002

3. METHOD AND IMPLEMENTATION

Images are resized to 224 × 224 RGB and normalized using ImageNet channel means and standard deviations. Training augmentation (horizontal flip probability 0.5 and random rotation within ±10°) is enabled for the regularized and VGG-16 experiments only; validation and test images are not randomly augmented. Seeds are fixed for reproducibility.

MODEL / STAGE	IMPLEMENTATION
VGG-6 baseline	Four convolutional blocks (32, 64, 128, 128 filters), 3 × 3 kernels and 2 × 2 max pooling; Dense(512) and one-logit binary output. No dropout or augmentation.
Regularized VGG-6	Same architecture with Dropout(0.5) and training augmentation to reduce overfitting and improve generalization.
Frozen VGG-16	ImageNet-pretrained convolutional features are frozen; a replacement Dense(512) / Dropout(0.5) binary head is trained.
Fine-tuned VGG-16	Blocks 1–4 stay frozen; block 5 and the classifier are trained for 10 additional epochs using learning rate 1×10^{-5}.

All experiments use RMSprop, batch size 32, and binary cross-entropy with logits. Training is run in the notebook; scripts handle data preparation, loading, model definitions and Grad-CAM. Checkpoints are evaluated on the same held-out test set.

4. RESULTS AND DISCUSSION

Each result below is measured on the same 1,002-image test set, with a 0.5 probability threshold. Macro precision, recall and F1 are averaged equally across the cat and dog classes.

Experiment	Test accuracy	Macro F1	Test loss	Correct / total
VGG-6 baseline	87.13%	0.8712	0.2926	873 / 1,002
VGG-6 + regularization	92.91%	0.9291	0.1853	931 / 1,002
Frozen VGG-16	98.70%	0.9870	0.0322	989 / 1,002
Fine-tuned VGG-16	**99.50%**	**0.9950**	**0.0242**	**997 / 1,002**

Interpretation and model selection

The baseline VGG-6 overfits: its training accuracy reached 99.59%, while final validation accuracy was 86.17%. Adding dropout and augmentation raised test accuracy from 87.13% to 92.91%. Using ImageNet-pretrained VGG-16 features produced a further jump to 98.70%, showing the value of transfer learning. Fine-tuning block 5 improved the measured test accuracy to 99.50%.

Deployment decision. The deployed model is the frozen VGG-16 checkpoint from epoch 2, selected before test evaluation because it had the lowest validation loss (approximately 0.0428). Fine-tuning did not improve validation loss, so its higher test accuracy was reported as a comparison rather than used to select the deployed model. This avoids choosing a model using test results.

5. EXPLAINABILITY AND DEMONSTRATION

Grad-CAM is applied to the final convolutional layer of VGG-16 (features[29]). The implementation captures feature maps and their gradients, averages gradients spatially to obtain channel weights, forms a weighted map, applies ReLU, resizes it to the input resolution, and normalizes it for display. For the binary output, the dog score is the logit and the cat score is the negative logit. The heatmap is a qualitative indication of influential regions, not a segmentation mask or proof of causation.

The Streamlit application (app.py) accepts JPG, JPEG, PNG and WebP uploads, displays the predicted class and cat/dog probability scores, and can generate a heatmap with an overlay for download as PNG. Inference supports GPU when available and falls back to CPU. A thread lock protects the shared model while Grad-CAM hooks are active.

6. CONCLUSION AND LIMITATIONS

The experiments show a clear progression: regularization improves the scratch-trained CNN, transfer learning delivers the largest gain, and fine-tuning gives a smaller additional test-set improvement. The project combines evaluation with a usable demo and visual explanations. Results should be interpreted with care: only one random seed/split was evaluated, near-duplicate images were not checked, performance on external datasets has not been established, and the binary classifier assigns either “cat” or “dog” even to unrelated images. Probability scores are not calibrated confidence guarantees.

REFERENCES AND PROJECT REPOSITORY

[1] Lei Lin, *Household Animals Classification Using Deep Learning*, Stanford CS229, Spring 2020 (reference report included with the project). [2] Kaggle, Dogs vs. Cats dataset. [3] Simonyan and Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition. [4] Selvaraju et al., Grad-CAM

Source code: github.com/Adarsh-031/household-animals-classification-using-deeplearning | Main artifacts: training.ipynb, scripts/prepare_data.py, scripts/data.py, scripts/models.py, scripts/explain.py and app.py.